Good Evening Everyone :)

Starting at 9:05pm

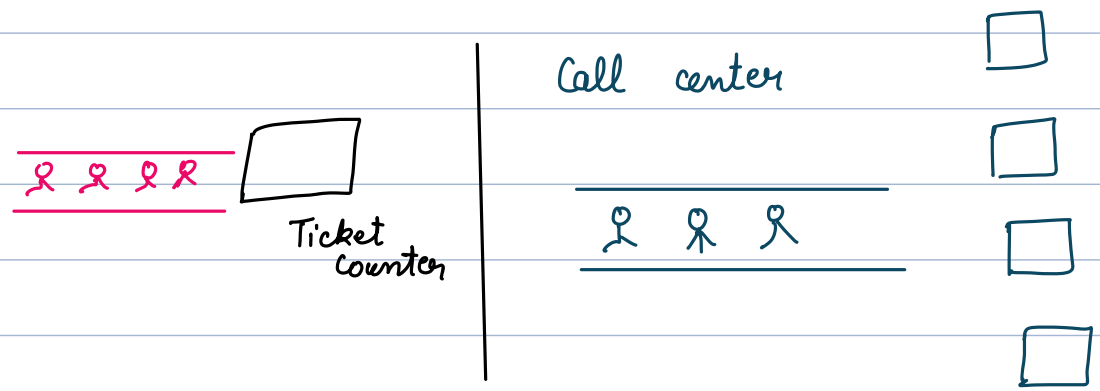
Agenda

- Queue
- Implementation of queue using Array
- Implementation of queue using stacks
- Doubly ended Queue
- Sliding Window Maximum.

Queue

A queue represent a linear data structure where items are added at one end and removed from other end. It follows FIFO principle.

Example



Common operations in queue include

1) Enqueue :- The operation adds an element to back of queue,

2) Dequeue → The operation removes and return element from the front of queue.

3) Peek or Front : look element at front without removing it

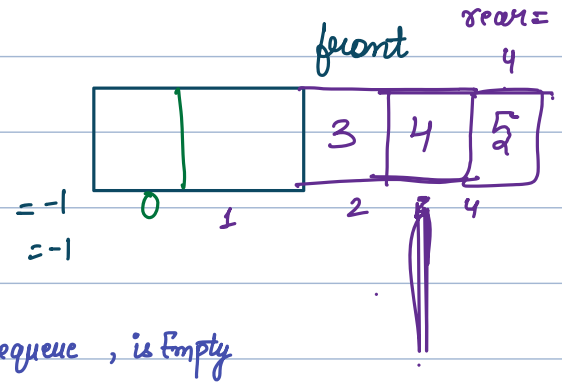
4) isEmpty → checks if queue is empty

5) Size or length : return size of queue.

6) isFull() : True / false.

Implementation of Queue

using dynamic array :-



Functions to implement : enqueue, dequeue, isEmpty

Enqueue

enqueue (element) :

if rear is -1 // Queue is empty

rear = 0 , front = 0

else if rear is end of array..

```
if ( front > 0) {
    # Move elements to the beginning of array
    for (i -> front to rear) {
        arr [i - front] = arr [i]
    }
    Set rear = rear - front
    Set front = 0
}
```

else

newsize = current_array size * 2.

create a new array.

copy all elements from old array to new array.

}

}

array [rear] = element

Increment rear by 1.

dequeue ()

```
if front is -1 {
    return Queue is Empty.
}
else
    element = arr [front]
    increment front by 1.

    if (front > rear)

        front = -1 , rear = -1
}

return element
```

isEmpty()

```
if front is -1
    return true
else
    return false.
```

Implementation of Queues using linked list

1) Insert at Head

TC :- O(1)

2) Insert at Tail

TC : O(N)

3) Delete at Head

TC : O(1)

4. Delete at Tail

O(N)

① Insertion at Head and Deletion at tail

② Insertion at tail and deletion at Head

Quiz1

Enq (3), Enq(7), Enq(12), deque(), dqueue (), Enq(8), Enq(3)

Front ← 12 8 3 ←

3 7

Quiz2.

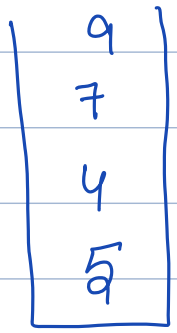
enqueue (4), dequeue(), enqueue (9), enqueue (8), enqueue (7), enqueue(11), enqueue (20), dqueue ()

front ← 2 7 11 20 ← rear/back.

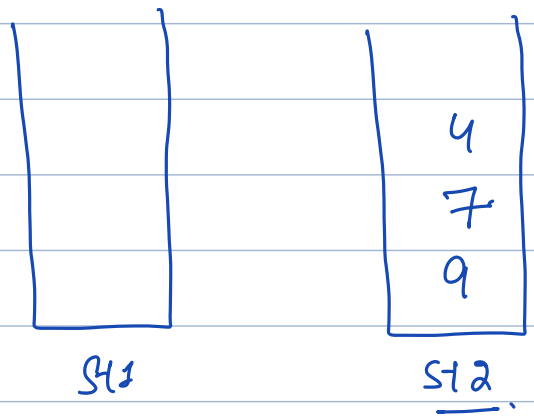
4 9

PROBLEM 1 : Implementation of queue using Stack

5 4 7 9 deque 8 10 Deq Deq.



Idea 1.

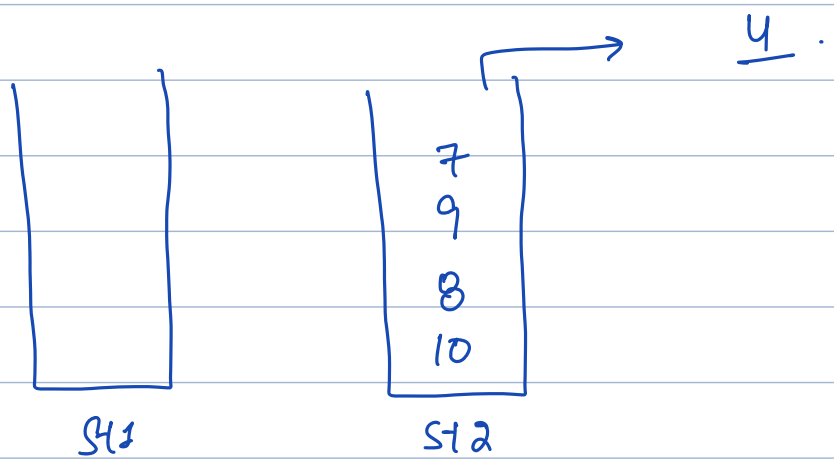


5

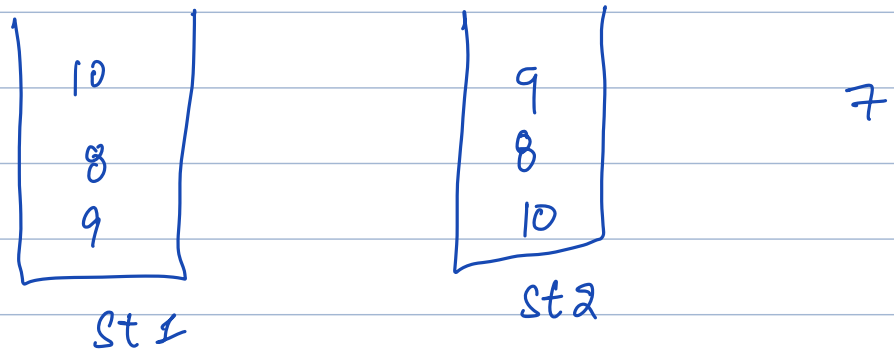
Step 1 → Pop out all elements from Stack 1 to Stack 2.

Step 2 → Pop top element out from stack 2.

Step 3 → Pop all elements from stack 2 to stack 1.



→

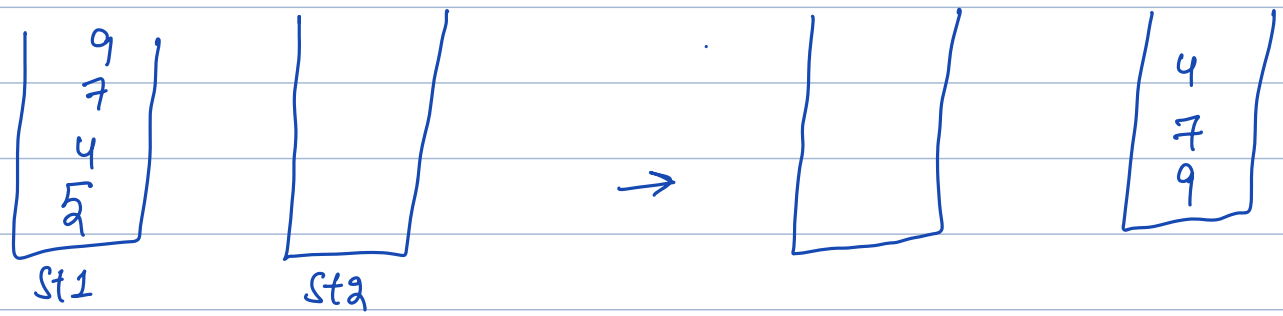


Deque.

Idea 2.

5 4 7 9 8

5 4 7 9 deque 8 10 deq deq 14 deq deq 21



Approach

For Enqueue → O(1)

Push x in Stack 1.

For Deque Best case : O(1) Worst: O(N).

1. If Stack 2 is empty

Transfer all elements Stack 1 → Stack 2.

2. Remove from Top of Stack 2

Doubly Ended Queue

data	next

→ Single linked

prev	data	next

← DLL.

A double ended queue is a data structure that allows element to be added or removed from both ends, making it versatile for various operation
A doubly linked list is a common choice for implementing a deque because it provides efficient operations for adding and removing element from both sides.

Insertion at Front : insert an element on the front (head) of the deque.

Insertion at back : Add an element to the back (tail) of deque.

Removal from Front : Removal of an element from front of the deque.

Removal from back : Removal of an element from back of the deque.

Front Element Access → get the front element

Rear Element Access. → get the back/ Rear element.

PROBLEM 2 : Sliding Window Maximum

Given an integer array A and an window of size k, find max element.

Input

An integer representing length of subarray A in which average of all elements greater than or equal to k. If no such subarray exists, return 0.

Example

A = [1 8 5 6 7 $\boxed{4 \quad 2 \quad 0 \quad 3}$] K = 4.

8 8 7 7 7 4.

Quiz 3.

A[] = [1 4 $\boxed{3 \quad 2 \quad 5}$] K = 3

4 4 5

Brute Force :- find maximum in each window of size, which means you need access each element k times

TC: O(n*k) times

$A = [1 \quad 8 \quad 5 \quad 6 \quad 7 \quad 4 \quad 2 \quad 0 \quad 3]$ $K=4$

indices: 0 1 2 3 4 5 6 7 8 (i)

Deq.

→ ~~1~~ ~~8~~ ~~5~~ ~~6~~ ~~7~~ 4 ~~2~~ ~~0~~ 3 ←

Code :-

TC : O(N)

SC : O(N)

```
func. findMaxInGroups (A, K)

n = length (A)

for (i → 0 to K-1) {

    while (!q.isEmpty() and A[i] >= A[q.rear()]) {
        q.dequeRear()
    }

    q.enqueRear (i)
}

print A[q.front()]      //Print max for first group.
```

```
// Slide for next window

for (i -> k to n-1)
{
    if (!q.isEmpty and q.front() == i-k)
    {
        q.dequeFront()
    }

    while (!q.isEmpty and A[i] >= A[q.rear()]){
    {
        q.dequeRear()
    }
    q.enqueRear(i)

    print(A[q.front()])
}
```

$A = [\underset{0}{2} \quad 4 \quad 6 \quad 1 \quad 8 \quad 9 \quad 7 \quad 5 \quad \overset{i}{3}]$

$K = 3$

Expected $\Rightarrow$ 6 6 8 9 9 9 7

6 6 8 9 9 9 7

~~2~~ ~~4~~ ~~6~~ ~~1~~ ~~8~~ ~~9~~ 7 5 3

Scenario:

At TechTrade Inc., a trading team focuses on day-trading technology stocks. They employ a strategy that involves selling stocks at short-term peaks to maximize profits. The team uses an algorithm to determine the best time to sell stocks based on minute-to-minute price data.

Stock Prices Array A

Consider the minute-by-minute stock prices of a tech company, TechCorp, over a 10-minute interval:

A=[220,215,230,225,240,235,230,245,250,240] ⇒ 9:00 - 9:10AM

9:00 → 9:02